

Universidad de Sevilla
Escuela Técnica Superior de Ingeniería Informática

Documentación de la entrega 02

Documentación Análisis del Código Fuente y Métricas Asociadas



Grado en Ingeniería Informática – Ingeniería del Software

Proceso y Gestión Software 2

Curso 2023 – 2024

Fecha	Versión
30/03/2024	v1r1

Grupo de prácticas: G5-54

Autores	Rol	Correo electrónico
Blanco Mora, David	Desarrollador	davblamor@alum.us.es
García Lama, Gonzalo	Scrum Manager	gongarlam@alum.us.es
Martín de Prado Barragán, Carlos	Desarrollador	carmarbar9@alum.us.es
Juan Núñez Sánchez	Desarrollador	juanunsan2@alum.us.es
Jiménez Soriano, Lidia	Desarrollador	lidjimsor@alum.us.es
Yao, Jun	Desarrollador	junyao@alum.us.es

Repositorio: <https://github.com/gii-is-psg2/psg2-2324-g5-54>

Índice de contenido

1. Introducción 2

2. Control de Versiones 3

3. Contenido 3

Capturas SonarQube 3

Backend 4

FrontEnd 8

Información sobre Issues 12

Backend 12

Frontend 12

4. Descripción y análisis de bugs potenciales 13

Bugs Potenciales en Backend 13

1# Tipo de Bug 13

2#Tipo de Bug 13

Bugs Potenciales en Frontend 14

3# Tipo de Bug 14

4# Tipo de Bug 14

5# Tipo de Bug 15

6# Tipo de Bug 15

6. Descripción y análisis de Vulnerabilidades 15

7. Descripción y análisis de los code smells 16

1# Code smell 16

2# Code smell 16

3# Code smell 17

4# Code smell 17

5# Code smell 17

6# Code smell 18

7# Code smell 18

8# Code smell 19

9# Code smell 19

10# Code smell 20

11# Code smell 20

12# Code smell 20

13# Code smell 21

7.1. Code Smell Solutions

8. Conclusiones 21

1. Introducción

En este documento se analiza el código fuente del proyecto para detectar imperfecciones mediante la herramienta SonarQube. Tras obtener los resultados, se hace un análisis de los mismos.

2. Control de Versiones

Fecha	Versión	Descripción
27/03/2024	v1r0	Inicialización del documento
30/03/2024	v1r1	Finalización del documento

3. Contenido

Capturas SonarQube

Backend

QUALITY GATE STATUS ⓘ

MEASURES

Passed

All conditions passed.

New Code

Overall Code

13

Bugs

Reliability

C

0

Vulnerabilities

Security

A

1

Security Hotspots ⓘ

0.0%

Reviewed

Security Review

E

6h 44min

Debt

257

Code Smells

Maintainability

A

69.4%

Coverage on 1.6k Lines to cover

252

Unit Tests

0.5%

Duplications on 4.8k Lines

2

Duplicated Blocks



▼ Reliability ?				
Overview				
Overall				
Bugs	13			
Rating	C			
Remediation Effort	1h 35min			
▼ Security ?				
Overview				
Overall				
Vulnerabilities	0			
Rating	A			
Remediation Effort	0			
▼ Security Review ?				
Overall				
Security Hotspots	1			
Rating	E			
Security Hotspots Reviewed	0.0%			
▼ Maintainability ?				
Overview				
Overall				
Code Smells	257			
Debt	6h 44min			
Debt Ratio	0.3%			
Rating	A			
Effort to Reach A	0			
			▼ Coverage OVERVIEW	
			Overview	
			Overall	
			Coverage	69.4%
			Lines to Cover	1,612
			Uncovered Lines	477
			Line Coverage	70.4%
			Conditions to Cover	355
			Uncovered Conditions	124
			Condition Coverage	65.1%
			Tests	
			Unit Tests	252
			Errors	0
			Failures	0
			Skipped	0
			Success	100%
			Duration	5s
			▼ Duplications	
			Overview	
			Overall	
			Density	0.5%
			Duplicated Lines	31
			Duplicated Blocks	2
			Duplicated Files	1

▼ Size	
Lines of Code	4,750
Lines	6,797
Statements	1,266
Functions	412
Classes	101
Files	104
Comment Lines	302
Comments (%)	6.0%
▼ Complexity ?	
Cyclomatic Complexity	562
Cognitive Complexity	313
▼ Issues	
Issues	270
Open Issues	270
Reopened Issues	0
Confirmed Issues	0
False Positive Issues	0
Won't Fix Issues	0

	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
 main/java/org/springframework/samples/petclinic	4,460	13	0	236	1	69.4%	0.5%
 adoptions	352	10	0	14	0	10.8%	0.0%
 auth	233	0	0	8	0	86.7%	0.0%
 booking	222	0	0	15	0	38.2%	0.0%
 clinic	212	1	0	5	0	38.3%	0.0%
 clinicowner	143	1	0	5	0	33.3%	0.0%
 configuration	419	0	0	11	0	32.8%	0.0%
 consultation	604	0	0	49	0	90.9%	0.0%
 developers	27	0	0	4	1	0.0%	0.0%
 exceptions	168	0	0	13	0	92.7%	0.0%
 model	72	0	0	0	0	94.7%	0.0%
 owner	259	0	0	10	0	100%	0.0%
 pet	396	0	0	27	0	99.0%	0.0%
 pethotelroom	232	1	0	20	0	9.4%	10.4%
 user	281	0	0	13	0	94.7%	0.0%
 util	87	0	0	4	0	23.4%	0.0%
 vet	340	0	0	13	0	100%	0.0%
 visit	395	0	0	25	0	98.7%	0.0%
 PetclinicApplication.java	9	0	0	0	0	33.3%	0.0%
 PetclinicInitializer.java	9	0	0	0	0	0.0%	0.0%
	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
 main/java/org/springframework/samples/petclinic	4,460	13	0	236	1	69.4%	0.5%
 adoptions	352	10	0	14	0	10.8%	0.0%
 auth	233	0	0	8	0	86.7%	0.0%
 booking	222	0	0	15	0	38.2%	0.0%
 clinic	212	1	0	5	0	38.3%	0.0%
 clinicowner	143	1	0	5	0	33.3%	0.0%
 configuration	419	0	0	11	0	32.8%	0.0%
 consultation	604	0	0	49	0	90.9%	0.0%
 developers	27	0	0	4	1	0.0%	0.0%
 exceptions	168	0	0	13	0	92.7%	0.0%
 model	72	0	0	0	0	94.7%	0.0%
 owner	259	0	0	10	0	100%	0.0%
 pet	396	0	0	27	0	99.0%	0.0%
 pethotelroom	232	1	0	20	0	9.4%	10.4%
 user	281	0	0	13	0	94.7%	0.0%
 util	87	0	0	4	0	23.4%	0.0%
 vet	340	0	0	13	0	100%	0.0%
 visit	395	0	0	25	0	98.7%	0.0%
 PetclinicApplication.java	9	0	0	0	0	33.3%	0.0%
 PetclinicInitializer.java	9	0	0	0	0	0.0%	0.0%

QUALITY GATE STATUS ?

MEASURES

All conditions passed.

New Code

Overall Code

 BugsReliability

£

Security

5

0.0% Reviewed

Security Review 

Debt

•

Maintainability **A**

0.0%

Unit Tests



15.3%

Duplications on 12k Lines

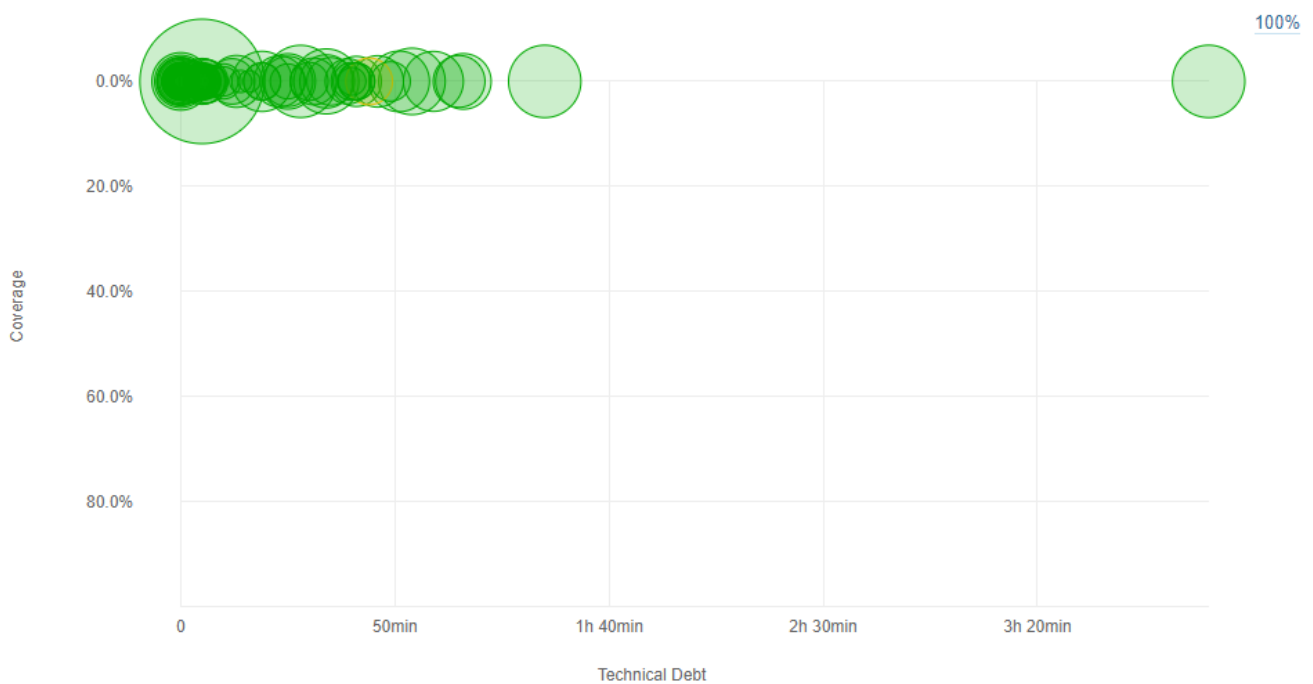
57

Duplicated Blocks

Risk ?

Color: Worse of Reliability Rating and Security Rating Size: Lines of Code

■ A
 ■ B
 ■ C
 ■ D
 ■ E



▼ Reliability ?	
Overview	
Overall	
Bugs	4
Rating	C
Remediation Effort	18min
▼ Security ?	
Overview	
Overall	
Vulnerabilities	0
Rating	A
Remediation Effort	0
▼ Security Review ?	
Overall	
Security Hotspots	1
Rating	E
Security Hotspots Reviewed	0.0%
▼ Maintainability ?	
Overview	
Overall	
Code Smells	156
Debt	3d 1h
Debt Ratio	0.4%
Rating	A
Effort to Reach A	0
▼ Complexity ?	
Cyclomatic Complexity	1,408
Cognitive Complexity	1,194
▼ Issues	
Issues	160
Open Issues	160
Reopened Issues	0
Confirmed Issues	0
False Positive Issues	0
Won't Fix Issues	0

▼ Coverage OVERVIEW	
Overview	
Overall	
Coverage	0.0%
Lines to Cover	2,576
Uncovered Lines	2,576
Line Coverage	0.0%
▼ Duplications	
Overview	
Overall	
Density	15.3%
Duplicated Lines	2,096
Duplicated Blocks	57
Duplicated Files	33
▼ Size	
Lines of Code	12,419
Lines	13,733
Statements	2,661
Functions	794
Classes	18
Files	139
Comment Lines	103
Comments (%)	0.8%

	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
src	12,382	4	0	156	1	0.0%	15.3%
__mocks__	2	0	0	0	0	0.0%	0.0%
admin	2,982	0	0	29	0	0.0%	23.9%
auth	339	1	0	3	0	0.0%	8.7%
classes/admin	1,680	0	0	7	0	0.0%	5.7%
clinicOwner	1,220	0	0	27	0	0.0%	51.3%
components/formGenerator	790	0	0	12	0	0.0%	0.0%
dashboard	26	0	0	0	0	0.0%	0.0%
home	14	0	0	0	0	0.0%	0.0%
hooks	6	0	0	1	0	0.0%	0.0%
mocks	272	0	0	12	0	0.0%	0.0%
owner	2,277	0	0	43	1	0.0%	9.3%
privateRoute	28	0	0	1	0	0.0%	0.0%
public	153	0	0	3	0	0.0%	59.9%
services	230	0	0	4	0	0.0%	0.0%
static/css	850	2	0	0	0	—	0.0%
util	177	0	0	3	0	0.0%	0.0%
validators	33	0	0	0	0	0.0%	0.0%
vet/consultations	445	0	0	10	0	0.0%	35.4%
App.css	36	0	0	0	0	—	0.0%
App.js	508	0	0	1	0	0.0%	0.0%
AppNavbar.js	201	0	0	0	0	0.0%	0.0%
AppNavbar.test.js	20	0	0	0	0	0.0%	0.0%
error-page.jsx	14	0	0	0	0	0.0%	0.0%
global.css	15	1	0	0	0	—	0.0%
index.css	12	0	0	0	0	—	0.0%
index.js	18	0	0	0	0	0.0%	0.0%
reportWebVitals.js	12	0	0	0	0	0.0%	0.0%
setupTests.js	5	0	0	0	0	0.0%	0.0%
test-utils.jsx	17	0	0	0	0	0.0%	0.0%

Información sobre Issues

Backend

▼ Type			
🐛 Bug			13
🔒 Vulnerability			0
👤 Code Smell			257
▼ Severity			
🛑 Blocker	0	🟢 Minor	51
🔴 Critical	3	🔵 Info	172
🔴 Major	44		

Frontend

▼ Type			
🐛 Bug			4
🔒 Vulnerability			0
👤 Code Smell			156
▼ Severity			
🔴 Blocker	0	🟢 Minor	28
🔴 Critical	32	🔵 Info	0
🔴 Major	100		

4. Descripción y análisis de bugs potenciales

Tenemos 13 bugs en Backend que se repiten y quedan en 2 realmente; por otro lado en el Frontend tenemos 4 bugs que se repiten por estilado de fuente y otros varios.

Bugs Potenciales en Backend

1# Tipo de Bug

☐

Call "Optional#isPresent()" before accessing the value. Why is this an issue?

11 days ago ▼ L32 🔗 🔍

🐛 Bug ▼ 🔴 Major ▼ 🔵 Open ▼ 🧑🏻‍💻 juanunsan ▼ 10min effort Comment

🔗 cwe ▼

Descripción: En la siguiente línea del método se ve lo siguiente:
adoptionRepository.findById(adoptionId).get();

Análisis: Se trata de un bug mayor, dado que la adopción puede no existir y por tanto hacer un get a un nulo y lanzar una excepción de NoSuchElementException.

Solución: Se solucionaría haciendo uso de .isPresent() en un if, donde si el valor existe se realiza el get.

2#Tipo de Bug

☐

Use the "equals" method if value comparison was intended. Why is this an issue?

11 days ago ▼ L42 🔗 🔍

🐛 Bug ▼ 🔴 Major ▼ 🔵 Open ▼ 🧑🏻‍💻 juanunsan ▼ 5min effort Comment

🔗 cert, cwe ▼

Descripción: Se usa el operador == para comparación de Integers en la línea de código perteneciente
if(offer.getOfferingOwner().getUser().getId() == userId)

Lo mismo sucede para otras líneas de otros ficheros:

Análisis: Se trata de un bug mayor, dado que este tipo de comparación no compara los valores almacenados en ambos campos de la comparación sino sus localizaciones en memoria. Por lo tanto, la evaluación es errónea.

Solución: Se solucionaría cambiando el operador == por equals.

Bugs Potenciales en Frontend

3# Tipo de Bug

☐ Unexpected duplicate "position" Why is this an issue?

2 months ago ▾ L103 🔗 🔍 ▾

☐ 🐛 Bug ▾ 📌 Major ▾ 🔓 Open ▾ Not assigned ▾ 1min effort Comment

🏷️ No tags ▾

Descripción: Se encontró un duplicado inesperado del identificador "position" en el código.

Análisis: El uso de un identificador duplicado puede llevar a confusiones y errores en el código. En este caso, la presencia de dos elementos con el mismo nombre puede causar ambigüedad y dificultar el mantenimiento y la comprensión del código.

Solución: Para resolver este problema, se debe renombrar uno de los elementos duplicados, asegurándose de que cada identificador sea único dentro del alcance relevante.

4# Tipo de Bug

☐ Unexpected duplicate "height" Why is this an issue?

2 months ago ▾ L13 🔗 🔍 ▾

☐ 🐛 Bug ▾ 📌 Major ▾ 🔓 Open ▾ Not assigned ▾ 1min effort Comment

🏷️ No tags ▾

Descripción: Se ha identificado un duplicado inesperado del identificador "height" en el código.

Análisis: La presencia de dos elementos con el mismo nombre, en este caso "height", puede generar confusión y dificultar la comprensión del código. Esto podría llevar a errores y dificultades durante el desarrollo y mantenimiento del software.

Solución: Para resolver este problema, se recomienda renombrar uno de los elementos duplicados, asegurándose de que cada identificador sea único dentro del contexto en el que se utiliza.

5# Tipo de Bug

Remove this conditional structure or edit its code blocks so that they're not all the same. Why is this an issue?

2 months ago ▾ L87 🔗 🔍 ▾

☐ 🐛 Bug ▾ 📌 Major ▾ 🔓 Open ▾ Not assigned ▾ 15min effort Comment

🏷️ No tags ▾

Descripción: Se ha detectado una estructura condicional redundante en el código.

Análisis: La estructura condicional actual presenta bloques de código idénticos en todas sus ramas. Esto sugiere una redundancia en la lógica de control de flujo y puede ser innecesario para el propósito previsto.

Solución: Para resolver este problema, se recomienda eliminar la estructura condicional si no es necesaria o modificar los bloques de código para que no sean todos iguales. Esto puede implicar ajustar las condiciones o introducir lógica adicional para diferenciar los casos.

6# Tipo de Bug

☐ Unexpected missing generic font family Why is this an issue?

2 months ago ▾ L16 🔗 🔍 ▾

☐ 🐛 Bug ▾ 📌 Major ▾ 🔓 Open ▾ Not assigned ▾ 1min effort Comment

🏷️ No tags ▾

Descripción:Se ha identificado una falta inesperada de la especificación de una familia genérica de fuentes en el código.

Análisis: La falta de especificación de una familia genérica de fuentes puede causar problemas de renderizado en diferentes entornos y dispositivos. Esto puede llevar a una apariencia inconsistente del texto en la interfaz

de usuario.

Solución: Para resolver este problema, se recomienda especificar una familia genérica de fuentes como respaldo en caso de que la fuente primaria no esté disponible. Esto ayuda a garantizar una apariencia consistente del texto en diferentes contextos de visualización.

6. Descripción y análisis de Vulnerabilidades

Las vulnerabilidades en el código pueden incluir problemas como inyección de código, XSS, CSRF, exposición de datos sensibles y desbordamiento de búfer. El análisis de vulnerabilidades implica identificar, clasificar y priorizar estas vulnerabilidades. Se utilizan herramientas de análisis estático y pruebas para este fin. Las vulnerabilidades se remedian mediante parches de seguridad y cambios en el diseño de la aplicación.

En nuestro caso no hemos encontrado ni se nos ha presentado errores de Vulnerabilidades tanto en Backend como Frontend.

7. Descripción y análisis de los code smells

1# Code smell

Declare this local variable with "var" instead. Why is this an issue?

 Code Smell ▾  Info ▾  Open ▾  juanunsan ▾ 0min effort [Comment](#)

Descripción: El code smell implica declarar explícitamente una variable local con un tipo específico cuando el tipo puede ser inferido de manera implícita usando la palabra clave var.

Causas: Prácticas de codificación habituales o convencionales que prefieren la declaración explícita de tipos.

Severidad: Afecta la legibilidad y mantenibilidad del código.

Solución: En lugar de declarar explícitamente el tipo, utiliza la palabra clave var para permitir que el compilador infiera el tipo según el valor asignado.

2# Code smell

Use the primitive boolean expression here. Why is this an issue?

 Code Smell ▾  Minor ▾  Open ▾  juanunsan ▾ 5min effort [Comment](#)

Descripción: Este code smell implica el uso de una expresión booleana primitiva (== false o == true) en lugar de utilizar directamente el valor booleano.

Causas: Prácticas de codificación habituales que hacen un uso excesivo de comparaciones booleanas explícitas.

Severidad: La severidad de este code smell es relativamente baja en términos de funcionalidad, pero puede afectar la legibilidad y la mantenibilidad del código. Utilizar una expresión booleana directa en lugar de una

comparación explícita puede hacer que el código sea más claro y menos propenso a errores.

Solución: En lugar de comparar con `== false`, simplemente use la negación `!` del valor booleano.

3# Code smell

Use isEmpty() to check whether the collection is empty or not. Why is this an issue?

 Code Smell  Minor  Open  juanunsan  2min effort [Comment](#)

Descripción: Este code smell consiste en usar `size() == 0` para verificar si una colección está vacía en lugar de utilizar el método `isEmpty()` proporcionado por la interfaz de la colección.

Causas: Costumbres heredadas de otros lenguajes de programación donde `size()` es la forma más común de verificar la longitud de una colección.

Severidad: Este code smell no suele tener un gran impacto en la funcionalidad del código, pero puede afectar la legibilidad y la eficiencia. Usar `isEmpty()` es más claro y expresa la intención de manera más directa que `size() == 0`.

Solución: Utilizar `isEmpty()`: En lugar de verificar `size() == 0`, utilizar el método `isEmpty()` proporcionado por las clases de colección en Java.

4# Code smell

Provide the parametrized type for this generic. Why is this an issue?

 Code Smell  Major  Open  Not assigned  5min effort [Comment](#)

Descripción: Este code smell se produce al no especificar el tipo parametrizado para un genérico en una declaración de método o clase, lo que puede llevar a la pérdida de seguridad de tipos y a la ambigüedad en el código.

Causas: Descuido o falta de atención al detalle al escribir código.

Severidad: Este code smell puede tener un impacto moderado en la legibilidad y la seguridad del código, especialmente en proyectos grandes y complejos donde la falta de especificación del tipo puede causar confusión y errores de compilación.

Solución: Especificar el Tipo Parametrizado: Proporciona el tipo parametrizado para el genérico en la declaración del método o clase.

5# Code smell

Replace this lambda with a method reference. Why is this an issue?

 Code Smell  Minor  Open  Not assigned  2min effort [Comment](#)

Descripción: Este code smell ocurre cuando se utiliza una lambda para realizar una operación simple que podría ser reemplazada por una referencia a método, lo que puede mejorar la legibilidad y la concisión del

código.

Causas: No aprovechar todas las características del lenguaje Java moderno.

Severidad: Este code smell es leve y principalmente afecta la legibilidad y la estética del código. Sin embargo, el uso de referencias a método puede hacer que el código sea más claro y conciso.

Solución: Utilizar una referencia a método en lugar de la lambda para operaciones simples.

6# Code smell

Replace the type specification in this constructor call with the diamond operator ("<>").

Why is this an issue?

 Code Smell  Minor  Open  Not assigned  1min effort [Comment](#)

Descripción: Este code smell ocurre cuando se especifica el tipo de datos en una llamada al constructor cuando se puede usar el operador diamante (<>) para inferir el tipo automáticamente.

Causas: Costumbres heredadas de versiones anteriores de Java donde el operador diamante no estaba disponible.

Severidad: Este code smell es leve y principalmente afecta la estética y la claridad del código. Sin embargo, el uso del operador diamante puede hacer que el código sea más limpio y conciso.

Solución: Reemplaza la especificación del tipo en la llamada al constructor con el operador diamante (<>).

7# Code smell

Remove this unused import 'org.springframework.web.bind.annotation.RequestParam'.

Why is this an issue?

 Code Smell  Minor  Open  Not assigned  2min effort [Comment](#)

Descripción: Importación no utilizada

Causas: Olvido o descuido al limpiar el código después de hacer cambios.

Severidad: Este code smell es leve y principalmente afecta la estética del código. Sin embargo, una acumulación de importaciones no utilizadas puede hacer que el código sea menos legible y mantener una base de código más grande de lo necesario.

Solución: Elimina la importación no utilizada del código.

8# Code smell

This block of commented-out lines of code should be removed. Why is this an issue?

 Code Smell  Major  Open  Not assigned  5min effort [Comment](#)

Descripción: Este code smell ocurre cuando se incluyen bloques de código comentados en el código fuente. Los bloques de código comentados pueden aumentar la confusión y dificultar la comprensión del código activo.

Causas: Falta de limpieza del código después de hacer cambios.

Severidad: Este code smell es leve y principalmente afecta la legibilidad y la estética del código. Los bloques de código comentados pueden hacer que el código sea más difícil de entender y pueden causar confusión, especialmente para los nuevos desarrolladores que trabajan en el proyecto.

Solución: Elimina completamente los bloques de código comentados del código fuente.

9# Code smell

Define a constant instead of duplicating this literal "No tienes permisos para realizar esta acción" 3 times. Why is this an issue?

 Code Smell  Critical  Open  Not assigned 8min effort [Comment](#)

Descripción: Este code smell ocurre cuando se utiliza el mismo literal en múltiples lugares del código en lugar de definirlo una vez como una constante y reutilizarlo en todo el código. Esto puede hacer que el código sea más propenso a errores y dificultar su mantenimiento.

Causas: Descuido o falta de atención al detalle.

Severidad: Este code smell es leve y principalmente afecta la legibilidad y la mantenibilidad del código. El uso repetitivo de literales puede hacer que el código sea menos claro y más difícil de mantener, especialmente si el literal necesita ser actualizado en el futuro.

Solución: Definir una constante para el literal y usarla en lugar de repetir el mismo texto en múltiples lugares del código.

10# Code smell

Add a nested comment explaining why this method is empty, throw an UnsupportedOperationException or complete the implementation. Why is this an issue?

 Code Smell  Critical  Open  Not assigned 5min effort [Comment](#)

Descripción: Este code smell ocurre cuando se define un constructor vacío en una clase sin comentarios que expliquen su propósito o razón para estar vacío. Esto puede llevar a confusión para otros desarrolladores que revisen el código y puede indicar una falta de mantenimiento o implementación incompleta.

Causas: Olvido de añadir comentarios explicativos durante el desarrollo.

Severidad: Este code smell puede tener un impacto leve en la legibilidad y la mantenibilidad del código. Aunque un constructor vacío puede ser válido en ciertos casos, la falta de explicación puede causar confusión sobre su propósito.

Solución: Añade un comentario dentro del constructor explicando la razón por la cual está vacío. Esto puede incluir información sobre la intención de implementar funcionalidad en una etapa posterior.

11# Code smell

Remove this useless assignment to variable "setClinicOwners". Why is this an issue?

 Code Smell  Major  Open  Not assigned  15min effort [Comment](#)

Descripción: Este code smell ocurre cuando se asigna un valor a una variable que no se utiliza en ninguna parte del código. Esto puede conducir a confusión y aumentar la complejidad del código sin ningún beneficio real.

Causas: Falta de atención al refactorizar o modificar el código existente.

Severidad: Este code smell es de baja gravedad, ya que no afecta directamente la funcionalidad o la estabilidad del código. Sin embargo, puede causar confusión y hacer que el código sea más difícil de entender para otros desarrolladores.

Solución: Simplemente eliminar la asignación a la variable si no se utiliza en ninguna parte del código.

12# Code smell

Extract this nested ternary operation into an independent statement. Why is this an issue?

 Code Smell  Major  Open  Not assigned  5min effort [Comment](#)

Descripción: Este code smell se presenta cuando se utiliza una operación ternaria anidada para tomar decisiones complejas. Esto puede hacer que el código sea difícil de entender y dar lugar a errores.

Causas: Falta de familiaridad con otras estructuras de control de flujo más legibles.

Severidad: Este code smell tiene una gravedad baja a moderada. Aunque no afecta directamente la funcionalidad del código, puede dificultar la comprensión y mantenimiento del mismo, especialmente para otros desarrolladores que lo revisen.

Solución: Divide la operación ternaria en declaraciones independientes para hacer el código más legible y modular.

13# Code smell

Immediately return this expression instead of assigning it to the temporary variable "savedBooking".

Why is this an issue?

 Code Smell  Minor  Open  Not assigned  2min effort [Comment](#)

Descripción: Este code smell se presenta cuando se asigna un valor a una variable temporal que no se utiliza más adelante en el código, especialmente cuando esta variable temporal se utiliza únicamente para almacenar el resultado de una expresión que se devuelve de inmediato.

Causas: Patrones de programación heredados de otros lenguajes donde la asignación temporal es más común.

Severidad: Este code smell tiene una gravedad baja. Aunque no afecta directamente la funcionalidad del código, puede dificultar la comprensión y el mantenimiento del mismo, especialmente para otros desarrolladores que lo revisen.

Solución: Devuelve la expresión directamente en lugar de asignarla a una variable temporal.

7.1#. Code Smell Solutions

12# Code smell

- Backend:

1# Use the primitive boolean expression here.

2# Remove the unnecessary boolean literal.

- Code Smell: adoptions/AdoptionController.java

```
@GetMapping("/petsIds")
public ResponseEntity<List<Integer>> findAllPetAdoptedIds() {
    List<Adoption> adoptions = adoptionService.findAll();
    List<Integer> petIds = new ArrayList<>();
    for (Adoption adoption : adoptions) {
        if(adoption.getIsAccepted() == false){
```

- Solution:

```
@GetMapping("/petsIds")
public ResponseEntity<List<Integer>> findAllPetAdoptedIds() {
    List<Adoption> adoptions = adoptionService.findAll();
    List<Integer> petIds = new ArrayList<>();
    for (Adoption adoption : adoptions) {
        if(!adoption.getIsAccepted()){
            Integer id = adoption.getPet().getId();
            petIds.add(id);
        }
    }

    return new ResponseEntity<>(petIds, HttpStatus.OK);
}
```

3# Declare this local variable with var instead.

- Code Smell: adoptions/AdoptionService.java

```
@Transactional
public Adoption createAdoption(Integer petId) throws DataAccessException {
    Adoption adoption = new Adoption();
```

- Solution:

```
@Transactional
public Adoption createAdoption(Integer petId) throws DataAccessException {
    var adoption = new Adoption();
    var pet = petService.findPetById(petId);
    var owner = pet.getOwner();
    adoption.setOwner(owner);
    adoption.setPet(pet);
    adoption.setIsAccepted(isAccepted:false);

    return adoptionRepository.save(adoption);
}
```

4# Declare this local variable with var instead.

- Code Smell: adoptions/AdoptionService.java

```
Pet pet = petService.findPetById(petId);
```

- Solution:

```
@Transactional
public Adoption createAdoption(Integer petId) throws DataAccessException {
    var adoption = new Adoption();
    var pet = petService.findPetById(petId);
    var owner = pet.getOwner();
    adoption.setOwner(owner);
    adoption.setPet(pet);
    adoption.setIsAccepted(isAccepted:false);

    return adoptionRepository.save(adoption);
}
```

5# Declare this local variable with var instead.

- Code Smell: adoptions/AdoptionService.java

```
Owner owner = pet.getOwner();
```

- Solution:

```
@Transactional
public Adoption createAdoption(Integer petId) throws DataAccessException {
    var adoption = new Adoption();
    var pet = petService.findPetById(petId);
    var owner = pet.getOwner();
    adoption.setOwner(owner);
    adoption.setPet(pet);
    adoption.setIsAccepted(isAccepted:false);

    return adoptionRepository.save(adoption);
}
```

6# Declare this local variable with var instead.

- Code Smell: adoptions/OfferController.java

```
@PutMapping("/update/{offerId}")
public ResponseEntity<Offer> updateOffer(@RequestBody String status, @PathVariable("offerId") Integer offerId) {
    Offer offer = offerService.updateOffer(status, offerId);
}
```

- Solution:

```
@PutMapping("/update/{offerId}")
public ResponseEntity<Offer> updateOffer(@RequestBody String status, @PathVariable("offerId") Integer offerId) {
    var offer = offerService.updateOffer(status, offerId);
    return new ResponseEntity<>(offer,HttpStatus.OK);
}
```

7# Use the primitive boolean expression here.

8# Remove the unnecessary boolean literal.

- Code Smell: adoptions/OfferController.java

```
@DeleteMapping("/{petId}")
public ResponseEntity<Adoption> deleteAdoption(@PathVariable("petId") Integer petId) {
    List<Adoption> adoptions = adoptionService.findAdoptionByPetId(petId);
    for(Adoption adoption : adoptions){
        if(adoption.getIsAccepted() == false){

```

- Solution:

```
@DeleteMapping("/{petId}")
public ResponseEntity<Adoptation> deleteAdoptation(@PathVariable("petId") Integer petId) {
    List<Adoptation> adoptations = adoptionService.findAdoptationByPetId(petId);
    for(Adoptation adoption : adoptations){
        if(!adoption.getIsAccepted()){
            adoptionService.deleteAdoptation(adoption);
        }
    }
    return new ResponseEntity<>(HttpStatus.OK);
}
```

9# Call "Optional#isPresent()" before accesing the value".

- Bug: adoptions/OfferService.java

```
@Transactional(readonly = true)
public Offer findOfferById(int offerId) throws DataAccessException {
    return offerRepository.findById(offerId).get();
}
```

- Solution:

```
@Transactional(readonly = true)
public Offer findOfferById(int offerId) throws DataAccessException {
    Optional<Offer> offerOptional = offerRepository.findById(offerId);

    if (offerOptional.isPresent()) {
        return offerOptional.get();
    } else {
        throw new NoSuchElementException("Offer not found for ID: " + offerId);
    }
}
```

10# Use the "equals" method if value comparison was intended.

- Bug: adoptions/OfferService.java

```

@Transactional(readonly = true)
public List<Adoptation> findAllAdoptationNotOffered(Integer userId) throws DataAccessException {
    List<Adoptation> all = adoptionService.findAll();
    List<Offer> allOffers = offerRepository.findAll();
    for(Offer offer : allOffers){
        if(offer.getOfferingOwner().getUser().getId() == userId){
            all.remove(offer.getAdoptation());
        }
    }

    for(Adoptation adoption : all){
        if(adoption.getOwner().getUser().getId() == userId){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
        if(adoption.getIsAccepted() == true){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
    }
    return all;
}

```

- Solution:

```

@Transactional(readonly = true)
public List<Adoptation> findAllAdoptationNotOffered(Integer userId) throws DataAccessException {
    List<Adoptation> all = adoptionService.findAll();
    List<Offer> allOffers = offerRepository.findAll();
    for(Offer offer : allOffers){
        if(offer.getOfferingOwner().getUser().getId().equals(userId)){
            all.remove(offer.getAdoptation());
        }
    }

    for(Adoptation adoption : all){
        if(adoption.getOwner().getUser().getId().equals(userId)){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
        if(adoption.getIsAccepted() == true){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
    }
    return all;
}

```

11# Use the "equals" method if value comparison was intended.

- Bug: adoptions/OfferService.java

```

@Transactional(readonly = true)
public List<Offer> findAlloffersByUserId(Integer userId) throws DataAccessException {
    List<Offer> all = offerRepository.findAll();
    List<Offer> res = new ArrayList<>();
    for(Offer offer : all){
        if(offer.getOfferingOwner().getUser().getId() == userId){
            res.add(offer);
        }
    }
    return res;
}

@Transactional(readonly = true)
public List<Offer> findAlloffersReceived(Integer userId) throws DataAccessException {
    List<Offer> all = offerRepository.findAll();
    List<Offer> res = new ArrayList<>();
    for(Offer offer : all){
        if(offer.getAdoption().getOwner().getUser().getId() == userId && offer.getStatus().equals(AdoptionStatus.PENDING)){
            res.add(offer);
        }
    }
    return res;
}

```

- Solution:

```

@Transactional(readonly = true)
public List<Adoption> findAllAdoptionNotOffered(Integer userId) throws DataAccessException {
    List<Adoption> all = adoptionService.findAll();
    List<Offer> alloffers = offerRepository.findAll();
    for(Offer offer : alloffers){
        if(offer.getOfferingOwner().getUser().getId().equals(userId)){
            all.remove(offer.getAdoption());
        }
    }

    for(Adoption adoption : all){
        if(adoption.getOwner().getUser().getId().equals(userId)){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
        if(adoption.getIsAccepted() == true){
            all.remove(adoption);
            if(all.size() == 0){
                return new ArrayList<>();
            }
        }
    }
    return all;
}

```

12# Call "Optional#isPresent()" before accessing the value".

- Bug: adoptions/OfferService.java

```

@Transactional
public Offer createOffer(OfferDTO offerDTO) throws DataAccessException{
    Offer offer = new Offer();
    offer.setOfferingOwner(ownerService.optFindOwnerByUser(offerDTO.getOfferingOwnerUserId()).get());

    offer.setStatus(AdoptationStatus.PENDING);
    offer.setDescription(offerDTO.getDescription());
    offer.setAdoptation(adoptationService.findAdoptationById(offerDTO.getAdoptationId()));
    return offerRepository.save(offer);
}

```

- Solution:

```

public Offer createOffer(OfferDTO offerDTO) throws DataAccessException {
    Offer offer = new Offer();
    Optional<Owner> ownerOptional = ownerService.optFindOwnerByUser(offerDTO.getOfferingOwnerUserId());

    if (ownerOptional.isPresent()) {
        offer.setOfferingOwner(ownerOptional.get());
        offer.setStatus(AdoptationStatus.PENDING);
        offer.setDescription(offerDTO.getDescription());
        offer.setAdoptation(adoptationService.findAdoptationById(offerDTO.getAdoptationId()));
        return offerRepository.save(offer);
    } else {
        throw new NoSuchElementException("Owner not found for user ID: " + offerDTO.getOfferingOwnerUserId());
    }
}

```

13# Use the "equals" method if value comparison was intended.

- Bug: adoptions/OfferService.java

```

@Transactional
public void rejectAllOffersExcept1(Offer offer) throws DataAccessException{
    List<Offer> all = offerRepository.findAll();
    for(Offer o : all){
        if(o.getAdoptation().getId() == offer.getAdoptation().getId() && o.getId() != offer.getId()){
            o.setStatus(AdoptationStatus.REJECTED);
            saveOffer(o);
        }
    }
}

```

- Solution:

```

@Transactional
public void rejectAllOffersExcept1(Offer offer) throws DataAccessException{
    List<Offer> all = offerRepository.findAll();
    for(Offer o : all){
        if(o.getAdoptation().getId().equals(offer.getAdoptation().getId()) && !o.getId().equals(offer.getId())){
            o.setStatus(AdoptationStatus.REJECTED);
            saveOffer(o);
        }
    }
}

```

14# Use the "equals" method if value comparison was intended.

- Bug: adoptions/OfferService.java

```
@Transactional
public void deleteAdoptation(Adoptation adoption) throws DataAccessException {
    List<Offer> all = offerRepository.findAll();
    for(Offer offer : all){
        if(offer.getAdoptation().getId() == adoption.getId()){
            deleteOffer(offer);
        }
    }
    adoptionService.deleteAdoptation(adoption);
}
```

- Solution:

```
@Transactional
public void deleteAdoptation(Adoptation adoption) throws DataAccessException {
    List<Offer> all = offerRepository.findAll();
    for(Offer offer : all){
        if(offer.getAdoptation().getId().equals(adoption.getId())){
            deleteOffer(offer);
        }
    }
    adoptionService.deleteAdoptation(adoption);
}
```

15# Define a constant instead of duplicating this literal "No tienes permisos para realizar esta acción" 3 times.

- Critical Bug: booking/BookingController.java

- Solution:

```
private static final String NOT_ALLOWED = "No tienes permisos para realizar esta acción";
```

16# This block of commented-out lines of code should be removed.

- Major Bug: petclinic/user/UserService.java

- Solution:

- Solo elimine lineas comentarias de varios archivos, pero pongo como referencia a este .java

- Frontend:

1# Remove this conditional structure or edit its code blocks so that they're not all the same.

- Bug: src/auth/register/index.js

```

useEffect(() => {
  if (type === "Owner" || type === "Vet") {
    if (registerFormOwnerInputs[5].values.length === 1){
      fetch("/api/v1/clinics")
      .then(function (response) {
        if (response.status === 200) {
          return response.json();
        } else {
          return response.json();
        }
      })
    }
  }
})

```

- Solution:

```

useEffect(() => {
  if (type === "Owner" || type === "Vet") {
    if (registerFormOwnerInputs[5].values.length === 1){
      fetch("/api/v1/clinics")
      .then(function (response) {
        if (response.status === 200) {
          return response.json();
        }
      })
    }
  }
})

```

2# Unexpected duplicate "position"

- Bug: src/static/css/owner/consultations.css

```

.checkbox-wrapper-10 .tgl-flip + .tgl-btn:after,
.checkbox-wrapper-10 .tgl-flip + .tgl-btn:before {
  display: inline-block;
  transition: all 0.4s ease;
  width: 100%;
  text-align: center;
  position: absolute;
  line-height: 2em;
  font-weight: bold;
  color: #fff;
  position: absolute;
}

```

- Solution:

```
.checkbox-wrapper-10 .tgl-flip + .tgl-btn:after,  
.checkbox-wrapper-10 .tgl-flip + .tgl-btn:before {  
  display: inline-block;  
  transition: all 0.4s ease;  
  width: 100%;  
  text-align: center;  
  position: absolute;  
  line-height: 2em;  
  font-weight: bold;  
  color: #fff;  
  top: 0;  
  left: 0;  
  -webkit-backface-visibility: hidden;  
  backface-visibility: hidden;  
  border-radius: 4px;  
}
```

3# Unexpected duplicate "height"

- Bug: src/static/css/owner/editPet.css

```
.edit-pet-form-container{  
  max-width: 600px;  
  height: 800px;  
  width: 90%;  
  height: 90%;  
}
```

- Solution:

```
.edit-pet-form-container{  
  max-width: 600px;  
  width: 90%;  
  height: 90%;  
  margin-top: 10%;  
}
```

8. Conclusiones

En conclusión, hemos detectado y analizado todos los bugs, vulnerabilidades y code smells de nuestro proyecto. Hemos entendido varios de nuestros errores al programar, trataremos de solucionarlos y esperamos no volver a repetirlos.

Respecto a la deuda técnica, ha incrementado en casi dos horas, cantidad que intentaremos reducir de cara al siguiente Sprint.